

TASVIRLARNI TAHLIL QILISH VA ANGLASH · Amaliy mashg'ulot (Practice)

15-amaliy mashg'ulot. Modelni joriy etish: o'lchov, quantization, pruning

2026–2027 o'quv yili

Mavzu. Latency (p50/p90), throughput, batch size; parametrlar, FLOPs va xotira; roofline; Amdahl va Little qonunlari, zanjir byudjeti; quantization (scale, zero-point, per-channel, int8); pruning (unstructured, structured), Conv-BN yig'ish, knowledge distillation; eksport (`torch.export`); takrorlanuvchanlik.

Maqsadlar. Talaba: (1) qatlamlar narxini, latency/throughput, Amdahl, scale va zero-point ni qo'lda hisoblaydi; (2) kvantlash xatosi, Conv-BN yig'ish va Amdahl formulalarini isbotlaydi; (3) `load_digits` da o'qitilgan kichik CNN ni kvantlaydi, kesadi, eksport qiladi va aniqlik, hajm hamda latency ni to'g'ri o'lchaydi.

Baholash (100 ball). A (1–11): $11 \times 2 = 22$; B (12–16): $5 \times 3 = 15$; C (17–30): $14 \times 4,5 = 63$. Kodda: to'g'ri ishlashi — 60%, chiqish formati va sonlar — 25%, tahlil — 15%.

Muhit. Python 3.11; `torch==2.14.0`, `numpy==2.4.4`, `scipy==1.17.1`, `scikit-learn==1.8.0`, `opencv-python-headless==4.13.0.92` (ONNX/onnxruntime ishlatilmaydi). Faqat CPU, `seed=42`, `torch.set_num_threads(1)`; ma'lumot `sklearn.datasets.load_digits` (kutubxona ichida). **Diqqat:** `torch 2.14` da `torch.quantization` — `DeprecationWarning`, `torch.jit.script/trace/save` — `FutureWarning` beradi; shuning uchun int8 qo'lda yoziladi, eksport — `torch.export`. Kod `python -W error::DeprecationWarning` da xatosiz ishlashi shart. **Vaqt** mashinaga bog'liq: vaqt chiqaradigan qatorlarga [TIME] belgisini qo'ying; qolgan qatorlar har ishga tushirishda bir xil bo'lishi kerak.

Hujjatlar. <https://pytorch.org/docs/stable/export.html>,
https://pytorch.org/docs/stable/generated/torch.nn.utils.prune.global_unstructured.html,
<https://pytorch.org/docs/stable/quantization.html>,
https://docs.python.org/3/library/time.html#time.perf_counter,
https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_digits.html,
https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.csr_matrix.html.

1. A bo'lim. Qo'lda hisoblash

- Latency o'lchovlari (ms): 3,2; 3,4; 3,1; 3,3; 9,8; 3,2; 3,5; 3,3; 3,1; 3,4. Mediana, o'rtacha, p90 (chiziqli interpolatsiya, `np.percentile`) va maksimum. O'rtacha medianadan necha foiz katta?
- Batch latency lari: $B = 1$: 1,2 ms; $B = 8$: 5,6; $B = 32$: 19,2; $B = 128$: 96 ms. Throughput va rasm uchun vaqt; eng yaxshi B ; 30 FPS kadr byudjetiga sig'adigan eng katta B ?
- Conv 3×3 , $32 \rightarrow 64$, chiqish 56×56 (bias bilan): parametrlar, MACs, FLOPs. Depthwise separable variant MACs i va tejash koeffitsiyenti ($1/C_{out} + 1/k^2$ bilan tekshiring).
- 8-bo'limdagi DigitsCNN (kirish $1 \times 8 \times 8$): har qatlam parametrlari va MACs, jami; fp32 hajmi (bayt). Faqat Linear og'irliklari int8 (per-channel fp32 scale, bias fp32) bo'lsa hajm va siqish koeffitsiyenti?
- Roofline: FC $1024 \rightarrow 512$, fp32, batch B . FLOPs = $2B \cdot 1024 \cdot 512$, baytlar = $4(1024 \cdot 512 + 1024B + 512B)$. $B = 1$ va $B = 64$ da intensivlik I ; $\pi = 200$ GFLOP/s, $\beta = 20$ GB/s mashinada memory- yoki compute-bound, erishiladigan unumdorlik va vaqt.
- Zanjir: dekodlash 6, preprocessing 4, model 18, postprocessing 2 ms. Model $3 \times$ tezlashsa umumiy tezlanish va chegarasi (Amdahl). 8 oqimda $5,2 \times$ o'lchangan: ketma-ket ulush f va 16 oqimdagi bashorat.
- 6-masaladagi zanjir: ketma-ket FPS; har bosqich alohida oqimda (pipeline) FPS va latency. Detector 40 ms, tracker 4 ms, detector har 5-kadrda: o'rtacha kadr vaqti va FPS.
- Affin int8 ($q \in [-128, 127]$), diapazon $[-0,5; 1,5]$: $s, z; r = 0,9, r = 2,0, r = 0$ uchun q va $\hat{r}$; xatoni $s/2$ bilan solishtiring. Simmetrik sxemada $r = 0,9$?
- 25,6 M parametrli model hajmi fp32/fp16/int8 (MiB). 100×80 fp32 matritsa 90% siyrak: zich va CSR (fp32 qiymat, int32 ustun indeksi, int32 `indptr`) hajmlari va nisbati.
- Little qonuni: $\lambda = 400$ so'rov/s, tizimda o'rtacha $W = 15$ ms. Bir vaqtdagi so'rovlar soni N ? Bitta nusxa bir vaqtda bitta so'rovni 15 ms da bajarsa, nechta nusxa kerak?
- Conv-BN yig'ish: $W = 0,8, b = 0,1, \gamma = 1,5, \beta = -0,2, \mu = 0,4, \sigma^2 = 0,25, \varepsilon = 0$: $W', b'; x = 1$ da tekshiring. Distillation: $z^t = (3, 1, 0)$ uchun $T = 1$ va $T = 3$ da `softmax(z/T)`.

2. B bo'lim. Isbot va tushuncha

- Conv qatlami MACs = $k^2 C_{in} C_{out} HW$ formulasini keltirib chiqaring; depthwise separable uchun nisbat $1/C_{out} + 1/k^2$ ekanini ko'rsating. Nega FLOPs latency ning to'liq bahosi emas (roofline, MaxPool)?
- Amdahl qonunini isbotlang; o'lchangan $S(n)$ dan f ni qanday baholash mumkin? Pipeline da nega

throughput eng sekin bosqich bilan, latency esa yigʻindi bilan aniqlanadi?

14. Diapazon ichida $|\hat{r} - r| \leq s/2$; asimmetrik sxemada $r = 0$ aynan ifodalanadi. Tekis taqsimlangan yaxlitlash xatosi dispersiyasi $s^2/12$ ekanidan har qoʻshimcha bit SQNR ni $\approx 6,02$ dB oshirishini koʻrsating. Nega per-channel scale yaxshiroq?
15. Conv-BN yigʻish formulasini isbotlang. Nega unstructured pruning zich formatda hajmni ham, CPU vaqtini ham kamaytirmaydi, structured pruning esa kamaytiradi? Nega `prune.remove` siz `state_dict` $\approx 2 \times$ katta?
16. Toʻgʻri vaqt oʻlchash qoidalari (warm-up, `perf_counter`, mediana/p99, navbatma-navbat oʻlchash, oqimlar). Band mashinada nega minimum barqarorroq? Nega seed model qurilishidan *oldin* oʻrnatilishi kerak?

3. C boʻlim. Dasturlash (PyTorch, torch-only)

Har topshiriq — alohida funksiya; `main()` ularni tartib bilan chaqiradi; chiqish `[raqam] ...` (vaqt qatorlari `[raqam][TIME] ...`). Umumiy kod:

```
import io, time, copy, numpy as np, cv2, torch, torch.nn as nn, torch.nn.functional as F
from sklearn.datasets import load_digits; from sklearn.model_selection import train_test_split
SEED = 42; torch.set_num_threads(1); cv2.setNumThreads(1)
def blok(i, o): return nn.Sequential(nn.Conv2d(i, o, 3, padding=1), nn.BatchNorm2d(o), nn.ReLU(), nn.
    MaxPool2d(2))
def baseline(): # 1x64x64 -> 3 sinf (ma'ruzadagi bazaviy model)
    return nn.Sequential(blok(1, 16), blok(16, 32), blok(32, 64), nn.Flatten(),
        nn.Linear(64 * 8 * 8, 128), nn.ReLU(), nn.Linear(128, 3)).eval()
class DigitsCNN(nn.Module):
    def __init__(self, c1=16, c2=32, h=64):
        super().__init__()
        self.conv1 = nn.Conv2d(1, c1, 3, padding=1); self.conv2 = nn.Conv2d(c1, c2, 3, padding=1)
        self.fc1 = nn.Linear(c2 * 2 * 2, h); self.fc2 = nn.Linear(h, 10)
    def forward(self, x):
        x = F.max_pool2d(F.relu(self.conv1(x)), 2); x = F.max_pool2d(F.relu(self.conv2(x)), 2)
        return self.fc2(F.relu(self.fc1(x.flatten(1))))
def digits(): # 1347 o'quv / 450 test
    d = load_digits(); X = (d.images / 16.0).astype(np.float32)[: , None]
    Xtr, Xte, ytr, yte = train_test_split(X, d.target, test_size=0.25, random_state=SEED, stratify=d.target)
    return torch.tensor(Xtr), torch.tensor(Xte), torch.tensor(ytr), torch.tensor(yte)
def train(model, Xtr, ytr, epochs=30, lr=1e-2, seed=SEED, loss_fn=None): # Adam, batch 64
    g = torch.Generator().manual_seed(seed); opt = torch.optim.Adam(model.parameters(), lr); model.train()
    for _ in range(epochs):
        perm = torch.randperm(len(Xtr), generator=g)
        for i in range(0, len(Xtr), 64):
            idx = perm[i:i + 64]; z = model(Xtr[idx])
            loss = F.cross_entropy(z, ytr[idx]) if loss_fn is None else loss_fn(z, idx)
            opt.zero_grad(); loss.backward(); opt.step()
    return model.eval()
def size_bytes(m): b = io.BytesIO(); torch.save(m.state_dict(), b); return len(b.getvalue())
def bench_many(fns, n=40, warmup=5): # navbatma-navbat o'lchash, har biri uchun minimum (ms)
    with torch.no_grad():
        for f in fns:
            for _ in range(warmup): f()
        t = []
        for _ in range(n):
            for k, f in enumerate(fns):
                t0 = time.perf_counter(); f(); t[k].append(time.perf_counter() - t0)
    return [1000 * min(v) for v in t]
```

17. `count_params_macs(model, x)` (forward hook: `conv — weight.numel()*H_out*W_out`, linear — `weight.numel()`). `baseline()` ($1 \times 1 \times 64 \times 64$) va `DigitsCNN()` uchun; 3- va 4-masala bilan solishtiring. *Chiqish:* [17] name: params=... MACs=... FLOPs~... M fp32=... KiB Linear params share=...%.
18. `bench(fn, n=100, warmup=20)` (mediana va p90, ms) bilan `baseline` ning $B = 1$ latency si; 1-masaladagi namunaning statistikasi. *Chiqish:* [18][TIME] baseline batch=1: median=... p90=... va [18] sample:
19. **Batch sweep.** $B \in \{1, 4, 16, 64\}$ (`randn`), `bench_many(n=30)`: latency, rasm uchun vaqt, throughput; eng yaxshi B va $B = 64/B = 1$ latency nisbati. *Chiqish:* [19][TIME] B=... latency=... ms
20. **FLOPs $\neq$ latency.** `baseline`, $B = 16$: MACs ulushi (Conv2d/Linear) va forward pre/post hook lar bilan qatlam turlari bo'yicha vaqt ulushi (30 takror). *Chiqish:* [20] MAC share: ... va [20][TIME] ... time

share:

21. **Use-case: bazaviy model.** `torch.manual_seed(42)`; DigitsCNN ni train bilan oʻqiting; test aniqligi va `size_bytes`. *Chiqish:* [21] `digits: train=... test=...; CNN test accuracy=... size=... bytes`.
22. `quant_params(rmin, rmax, bits, symmetric)` va `fake_quant` (NumPy): 8-masalani tekshiring (`[0.9, 0, -0.5, 2.0]`); `conv2` ogʻirliklari uchun SQNR: 8 va 4 bit per-tensor, 8 bit per-channel. *Chiqish:* [22] `sym=...; s=... z=... q=[...] r^=[...]` va SQNR qatorlari.
23. **Use-case: int8.** `QLinear`: ogʻirlik int8 (per-channel simmetrik, ± 127) bufer, aktivatsiya dinamik per-tensor int8, int32 matmul, soʻng `float·sxsw + b`. Ikkala `Linear` ni almashtiring; aniqlik, hajm, $\max|\Delta z|$, bashoratlar mosligi; test toʻplami (450) da latency. *Chiqish:* [23] `fp32: acc=... size=... | int8(Linear): acc=... size=... ratio=... max|dlogit|=... agree=... va [23] [TIME]`.
24. **Use-case: pruning.** Nusxalarda `prune.global_unstructured(L1Unstructured)` (4 qatlam ogʻirligi), sparsity 50, 70, 90, 95%: kesishdan keyin va 5 epoxa fine-tuning (`lr=1e-3`, mask saqlanadi) dan keyin aniqlik; `prune.remove` dan oldin/keyin hajm; `fc1` ning zich va CSR (`scipy.sparse`) baytlari. *Chiqish:* [24] `sparsity=...: acc prune=... -> fine-tune=...; zeros=...; size with mask=... B, dense ...; fc1 dense=... CSR=...`
25. **Structured pruning.** `conv2` filtrlarini L1 norma boʻyicha $32 \rightarrow 16$ (tartib saqlanadi), `fc1` kirish ustunlarini moslang (har kanal 4 ustun). Params, MACs, aniqlik (kesish, 5 epoxa fine-tuning), latency. *Chiqish:* [25] `conv2 32->16: params ... MACs ... acc ... -> cut ... -> fine-tune ... va [25] [TIME]`.
26. **Conv-BN yigʻish.** `baseline` (`manual_seed(42)`) BN statistiklari: `running_mean` $U(-0.5; 0.5)$, `running_var` $U(0.5; 2)$, γ $U(0.5; 1.5)$, β $U(-0.2; 0.2)$ (bloklar tartibida). Yigʻilgan model, $B = 16$ da $\max|\Delta|$, params, modullar soni, latency. *Chiqish:* [26] `Conv-BN fold: max|diff|=...; params ... -> ...; layers ... -> ... va [26] [TIME]`.
27. **Eksport.** `torch.export.export` (dinamik batch: `Dim('B', min=1, max=1024)`), `torch.export.save/load` (BytesIO); fayl hajmi, grafdagi `call_function` soni, chiqishlar farqi, batch 5 shakli, aniqlik, latency. *Chiqish:* [27] `torch.export: file=... B graph ops=...; ... va [27] [TIME]`.
28. **Distillation.** Oʻquvchi: `conv 1  $\rightarrow$  4  $\rightarrow$  8` (xuddi shu tuzilma), `Linear(32, 10)`. CE va KD ($T = 4$, $\alpha = 0.5$, $\alpha \text{CE} + (1 - \alpha)T^2\text{KL}$, oʻqituvchi — 21-topshiriq modeli) bilan seed 42, 43, 44 da (`manual_seed(seed)`, `train(..., seed=seed)`): aniqliklar, oʻrtacha, std. *Chiqish:* [28] `student CE/KD: seeds 42, 43, 44 acc=[...] mean=... std=... va parametrlar qatori`.
29. **Use-case: zanjir byudjeti.** Kadr: `default_rng(42).integers(0, 256, 480*640, uint8)` baytlari; bosqichlar: oʻqish (`frombuffer+copy`), `cv2.resize` (64×64 , `INTER_AREA`), `GaussianBlur` (3×3), tenzor (`/255`), `baseline`, `softmax+argmax`. Har bosqich vaqti va ulushi, p , Amdahl (model $2\times$), ketma-ket va pipeline FPS. *Chiqish:* [29] [TIME] ... (2 qator).
30. **Takrorlanuvchanlik.** DigitsCNN(8, 8, 32), 3 epoxa. (a) seed model qurilgandan keyin; (b) oldin. Har variant ikki marta. *Chiqish:* [30] `variant: run1=... run2=... reproducible=...`